

Integración de Servicios Cloud GCP

WSL, sitios estáticos, IAM y servicios de Google Cloud

Semana 07 - Sesión 14

Rel Guzman



**Universidad
Tecnológica
del Perú**

La sesión anterior

Vimos los servicios de despliegue más usados, **qué cobran** en su plan gratuito (cómputo, transferencia, builds y almacenamiento) y empezamos la ruta **Introducción a Google Cloud** en Google Skills.

Pregunta rápida

Tu página pesa 2 MB y la visitan 50 000 veces en un mes. ¿Cuántos GB de transferencia gastas?

Logro de la sesión

Al finalizar, podrás explicar para qué sirve **WSL**, publicar una landing en **GitHub Pages** y en **Vercel**, dar permisos con **IAM**, **habilitar un servicio** de Google Cloud y crear tu primera **máquina virtual**.

- ▶ Si usas Windows, ¿cómo practicarías comandos de Linux sin borrar tu sistema?
- ▶ ¿Se puede crear una máquina virtual **dentro** de otra máquina virtual?
- ▶ Si compartes tu proyecto de Google Cloud con un compañero, ¿le darías permiso para borrarlo todo?

Hoy juntas las piezas del día a día: trabajar con Linux desde Windows, publicar una página en minutos y entrar a Google Cloud sabiendo quién puede hacer qué y qué servicios están encendidos.

WSL

Linux dentro de Windows

Definición

WSL (*Windows Subsystem for Linux*) es una función de Windows que te deja usar **Linux real** (por ejemplo, Ubuntu) sin instalar otro sistema ni reiniciar la computadora.

Una comparación

Es como tener un **departamento dentro de tu casa**: comparte la luz y el agua (tu procesador y tu memoria), pero adentro las reglas son de Linux.

WSL 1 y WSL 2

WSL 2 es la versión actual: usa un núcleo (*kernel*) de Linux verdadero dentro de una máquina virtual muy ligera que Windows administra solo.

Instalarlo: una sola línea

En PowerShell como administrador:

```
wsl --install
```

Reinicias, eliges un usuario y contraseña de Linux, y listo: tienes Ubuntu.

Dónde están tus archivos

Tu disco C: aparece en Linux como `/mnt/c`. Tu carpeta de Linux aparece en el Explorador de Windows como *Linux*.

¿Para qué sirve? ¿Solo para practicar Linux?

Respuesta corta

Sirve para practicar, sí, pero sobre todo para **trabajar**: la mayoría de servidores en la nube usan Linux, y WSL te da ese mismo entorno en tu laptop con Windows.

Usos realistas

Programar en Python, Node.js o Go con las mismas herramientas que el servidor.

Docker Desktop en Windows usa WSL 2 por debajo.

Herramientas de nube: gcloud, ssh, git y scripts bash.

VS Code abre tus carpetas de Linux como si fueran locales.

Lo que no es

No es un servidor de producción: se apaga cuando cierras la laptop. Para publicar algo en Internet se usa la nube.

Consejo práctico

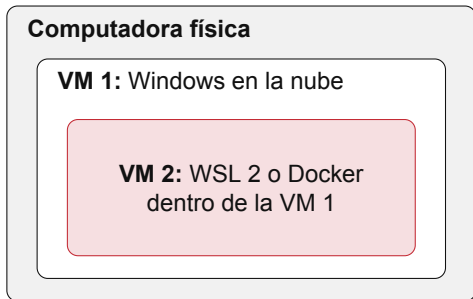
Guarda tus proyectos dentro de Linux (~ / proyectos), no en /mnt/c: es mucho más rápido.

Virtualización anidada

Una máquina virtual dentro de otra

Definición

Ejecutar una **máquina virtual dentro de otra máquina virtual**. Como las muñecas rusas (matrioskas): una dentro de otra.



Por qué no funciona siempre

Una VM necesita funciones especiales del procesador (*VT-x* en Intel, *AMD-V* en AMD). La VM 1 debe **recibir** esas funciones de la computadora física para poder crear la VM 2.

El costo

Cada capa agrega trabajo: la VM de adentro va **más lenta**.

Idea clave

No es algo de todos los días, pero aparece en situaciones muy concretas.

Casos reales

WSL o Docker en una VM: tu Windows es una VM (en la nube o en VirtualBox) y quieres usar WSL 2. WSL 2 ya es una VM, así que es anidada.

Laboratorios y cursos: cada alumno recibe una VM y dentro crea sus propias VMs.

Probar software de virtualización (como KVM) sin comprar servidores.

En Google Cloud

Compute Engine permite activarla al crear una VM Linux, con procesadores Intel y el hipervisor KVM.

Error típico

“WSL 2 requiere que la virtualización esté habilitada”. Si ves esto dentro de una VM, la anidada está apagada.

Despliegue de sitios estáticos

Primero GitHub Pages, luego Vercel

Definición

Un sitio hecho solo de **archivos**: HTML, CSS, JavaScript e imágenes. El servidor no calcula nada; solo **entrega los archivos** tal como están.

Una comparación

Estático: una carta impresa. Todos ven lo mismo.

Dinámico: un mozo que te pregunta y trae algo distinto a cada uno. Necesita backend y base de datos.

Quién hace el trabajo

En un sitio estático, el JavaScript corre en el **navegador del visitante**, no en un servidor tuyo.

Por eso es barato

No hay nada encendido esperando: GitHub Pages y Vercel lo publican gratis.

Idea clave

Es la landing “en construcción” de la **Práctica Calificada 1** (Sesión 10). Allí la servía `server.py`; hoy la dejamos en **solo tres archivos**.

Antes (Sesión 10)

```
ecommerce_webapp/  
  server.py  
  static/index.html  
  static/style.css
```

`server.py` armaba el enlace de WhatsApp.

Ahora (Sesion14/apps/landing)

```
landing/  
  index.html  <- contenido  
  styles.css  <- diseño  
  script.js   <- enlace WhatsApp
```

El enlace lo arma `script.js` en el navegador.

Lo que cambió en el HTML

Las rutas pasan de `/style.css` a `styles.css` (sin `/` al inicio). Así funciona en GitHub Pages, que publica en una subcarpeta.

Qué es

Un servicio **gratis** de GitHub que publica los archivos de un repositorio como página web.

Pasos

1. Crea un repositorio **público** y sube los tres archivos.
2. Entra a **Settings** → **Pages**.
3. En *Source* elige **Deploy from a branch**.
4. Elige la rama `main` y la carpeta `/` (`root`).
Guarda.
5. Espera uno o dos minutos y abre:
`https://usuario.github.io/repositorio/`

Límites del plan gratuito

Sitio de hasta **1 GB** y unos **100 GB** de transferencia al mes. Solo archivos estáticos.

Por qué `index.html`

Es el archivo que se abre cuando nadie pide uno en particular. Si se llama `inicio.html`, verás un error 404.

Y como el sitio vive en `/repositorio/`, una ruta `/styles.css` lo buscaría fuera: página sin estilos.

Qué es

Una plataforma PaaS muy usada para frontend. Se conecta a tu repositorio de GitHub y **publica sola** cada vez que haces `git push`.

Pasos

1. Entra a `vercel.com` con tu cuenta de GitHub.
2. **Add New** → **Project** e importa el repositorio.
3. En *Framework Preset* deja **Other**: no hay nada que compilar.
4. Presiona **Deploy**.
5. Abre `https://tu-proyecto.vercel.app`.

Lo que ganas

Cada rama o *pull request* recibe su propia dirección de **vista previa** (*preview*) para revisar antes de publicar.

Plan Hobby

Gratis, pero solo para proyectos **personales, no comerciales**. Unos 100 GB de transferencia al mes.

	GitHub Pages	Vercel
Qué publica	Solo archivos estáticos	Estáticos y también React, Next.js y funciones
Cómo se actualiza	Al hacer push a la rama elegida	Al hacer push a cualquier rama
Vista previa por rama	No	Sí
Dirección	<code>usuario.github.io/repo</code>	<code>proyecto.vercel.app</code>
Mejor para	Portafolios y documentación	Proyectos que van a crecer

Idea clave

Para tres archivos, las dos sirven igual. La diferencia aparece cuando el proyecto crece.

Pasos

1. Copia la carpeta `Sesion14/apps/landing` y ábrela: haz doble clic en `index.html`.
2. En `script.js` pon tu número de WhatsApp; en `styles.css` cambia el color `--accent`.
3. Súbela a un repositorio público nuevo de GitHub.
4. Publícala en **GitHub Pages**.
5. Impórtala en **Vercel**. Haz un cambio, un `git push` y mira cómo se actualizan las dos.

Si no se ven los estilos

Revisa que los nombres coincidan exactamente (`styles.css` no es `Styles.css`) y que la ruta no empiece con `/`.

Entregable

Las dos direcciones: `github.io` y `vercel.app`.

Idea clave

Una landing estática no guarda datos. Para probar un **backend** sin escribir un servidor, usamos **Figma Make** (con el plan educativo) conectado a **Supabase**.

Las dos piezas

Figma Education: el plan de pago de Figma, **gratis** para estudiantes y docentes verificados (figma.com/education).

Figma Make: describes una app con palabras y la genera funcionando.

Supabase: base de datos PostgreSQL y usuarios (login), lista para usar.

Pruébalo

1. Solicita el plan educativo con tu correo de la universidad.
2. En Figma Make pide: *“formulario de cotización de polos que guarde los pedidos”*.
3. Conecta Supabase cuando Figma lo pida y revisa la tabla creada.

Recuerda la Sesión 13

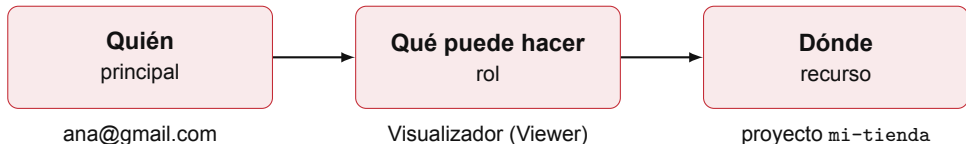
Supabase gratis: 500 MB de base de datos y el proyecto **se pausa** tras una semana sin uso.

Google Cloud IAM

Quién puede hacer qué, y dónde

Definición

IAM (*Identity and Access Management*) es el sistema de Google Cloud que decide **quién** puede hacer **qué** sobre **qué recurso**.



Una comparación

Un hotel: tu **tarjeta** (quién) abre tu habitación y el gimnasio (qué), pero no la cocina ni otras habitaciones.

Quién puede ser

Una cuenta de Google, un grupo o una **cuenta de servicio**: una “cuenta para programas”, por ejemplo una VM que lee archivos.

Qué es un rol

Un **rol** es un grupo de permisos con nombre. No das permisos sueltos: das roles.

Tres tipos

Básicos: *Propietario* (Owner), *Editor* y *Visualizador* (Viewer). Afectan a todo el proyecto.

Predefinidos: para un solo servicio. Ejemplo: *Visualizador de objetos de Storage* solo puede ver archivos.

Personalizados: los armas tú, permiso por permiso.

Mínimo privilegio

Da **solo lo necesario** para la tarea. Si alguien solo mira reportes, no necesita ser Editor.

Cuidado con Owner

El propietario puede borrar el proyecto y dar permisos a otros. Úsalo solo para quien administra.

Pasos

1. Entra a `console.cloud.google.com` y elige tu **proyecto** arriba.
2. Menú → **IAM y administración** → **IAM**.
3. Presiona **Otorgar acceso** (*Grant access*).
4. En *Principales nuevas* escribe el correo de tu compañero.
5. Elige un rol, por ejemplo **Visualizador**, y presiona **Guardar**.

Quitar el acceso

En la misma lista, el ícono del lápiz → eliminar el rol.

Si “no funciona”

Los cambios pueden tardar unos minutos en aplicarse. Espera antes de volver a intentar.

Habilitar servicios

Encender solo lo que vas a usar

¿Por qué hay que habilitar servicios?

Idea clave

En un proyecto nuevo, la mayoría de servicios están **apagados**. Antes de crear una VM o usar Cloud Run, debes **habilitar su API**.

Una comparación

Un departamento nuevo: la luz, el agua e Internet se **contratan** uno por uno. No pagas por el cable si no lo pediste.

Qué es una API aquí

La “puerta” por la que la consola, gcloud o tu programa le piden cosas a un servicio. Ejemplo: `compute.googleapis.com` es Compute Engine.

El error que vas a ver

“API has not been used in project ... before or it is disabled”. Traducción: enciéndela primero.

Antes de habilitar

Muchos servicios piden una **cuenta de facturación** vinculada, aunque uses el crédito gratis. Y necesitas un rol que lo permita, como Editor o Propietario.

Desde la consola

1. Menú → **APIs y servicios** → **Biblioteca**.
2. Busca **Compute Engine API**.
3. Presiona **Habilitar** y espera unos segundos.
4. Revísala en **APIs y servicios habilitados**.

Desde la terminal (Cloud Shell)

```
# Encender Compute Engine
gcloud services enable compute.
googleapis.com

# Ver lo que ya esta encendido
gcloud services list --enabled
```

Relación con IAM

IAM decide **quién** puede usar un servicio. Habilitar decide **si el servicio existe** en tu proyecto. Necesitas las dos cosas.

Google Skills: crea una máquina virtual

Todo lo anterior, en un laboratorio real

Qué es

Un laboratorio corto para principiantes en `skills.google/focuses/3563`. Creas una VM de Compute Engine (IaaS) y la conviertes en un servidor web.

Las tres tareas

1. Crear la VM `gcelab` desde la consola, marcando **Permitir tráfico HTTP**.
2. Entrar por SSH e instalar **NGINX**; abrir la IP externa en el navegador.
3. Crear una segunda VM, `gcelab2`, con `gcloud`.

Lo de hoy, en el lab

Compute Engine ya está **habilitado** y tu usuario temporal tiene un **rol de IAM** para crear VMs.

Los comandos clave

```
sudo apt-get update
sudo apt-get install -y nginx

gcloud compute instances \
  create gcelab2 \
  --machine-type=e2-medium \
  --zone=ZONA
```

Pasos

1. Entra a `skills.google/focuses/3563` y presiona **Start Lab**.
2. Abre la consola en una **ventana de incógnito** con el usuario temporal.
3. Revisa **IAM y administración** → **IAM**: ¿qué rol tiene tu usuario temporal?
4. Revisa **APIs y servicios**: ¿está habilitada Compute Engine API?
5. Sigue las tres tareas y presiona **Check my progress** en cada una.

Si la IP no abre

Usa `http://`, no `https://`, y revisa que marcaste *Permitir tráfico HTTP*.

Entregable

Captura de la página *Welcome to nginx!* y del lab completado.

Responde con tus palabras

1. Da un uso de WSL que no sea solo “practicar Linux”.
2. ¿Por qué WSL 2 dentro de una VM de Windows necesita virtualización anidada?
3. La landing de Estampa ya no necesita `server.py`. ¿Por qué? ¿Y por qué la tienda completa, con usuarios y pagos, sí necesitaría backend?
4. Un compañero solo necesita ver tu proyecto de Google Cloud. ¿Qué rol le das y por qué?
5. Aparece el error “*API ... is disabled*”. ¿Qué haces?

Lo que viene

Seguimos con la ruta de Google Skills y desplegamos en Google Cloud.

A vibrant watercolor splash background in shades of blue, purple, and pink, with the word "gracias" written in a white, cursive script across the center.

gracias



Universidad
Tecnológica
del Perú

- ▶ Microsoft Learn: *What is Windows Subsystem for Linux? y Install WSL*, learn.microsoft.com/windows/wsl.
- ▶ Microsoft Learn: *Comparing WSL 1 and WSL 2 y Working across file systems*.
- ▶ Google Cloud: *About nested virtualization*, documentación de Compute Engine, cloud.google.com/compute/docs.
- ▶ GitHub Docs: *About GitHub Pages y GitHub Pages limits*, docs.github.com/pages.
- ▶ Vercel Docs: *Deploying a project from Git y Hobby plan*, vercel.com/docs.
- ▶ Figma: *Figma for Education*, figma.com/education; Figma Help Center: *Connect a Figma Make file to Supabase*.
- ▶ Supabase: *Pricing y documentación*, supabase.com/docs.
- ▶ Google Cloud: *IAM overview y Roles and permissions*, cloud.google.com/iam/docs.
- ▶ Google Cloud: *Enabling and disabling services*, cloud.google.com/service-usage/docs.
- ▶ Google Skills: laboratorio *Crea una máquina virtual*, skills.google/focuses/3563.
- ▶ Las respuestas de todas las preguntas de esta sesión están en `respuestas.md`.